CIA Triad

CIA Triad — Cybersecurity Foundation:

The CIA Triad is a core model in cybersecurity that helps explain what secure systems are designed to protect.

CIA stands for **Confidentiality**, **Integrity**, and **Availability**, which are three principles used to evaluate whether information and systems are properly protected.

Understanding the CIA Triad matters because almost every security control, from passwords to backups, supports one or more of these principles.

Confidentiality:

Definition
Confidentiality means keeping information private and making sure only authorized people, systems, or applications can access it.
In cybersecurity, this often involves protecting sensitive data such as passwords, medical records, financial details, business documents, source code, payment information, or personal messages.

A system with strong confidentiality prevents attackers, unauthorized employees, or accidental viewers from seeing information they should not see.

Common ways to protect confidentiality include access controls, encryption, strong passwords, multi-factor authentication, and limiting permissions based on a user's role.

Confidentiality is important because leaked information can cause financial loss, identity theft, reputational damage, legal problems, or privacy violations.
For example, if a hospital exposes patient records, the issue is not only technical, it can also harm real people and violate privacy laws.

Confidentiality also applies to data in different states: data at rest, data in transit, and data in use.

- Data at rest means stored data, such as files on a hard drive
- Data in transit means data moving across a network
- Data in use means data currently being processed by a computer or application

How it's protected?

- **Encryption**: Converts readable data into unreadable form unless someone has the correct key.
- **Authentication**: Confirms a user's identity, usually through passwords, biometrics, security keys, or multi-factor authentication.
- **Authorization**: Controls what an authenticated user is allowed to access.
- **Least privilege**: Gives users only the minimum access needed to do their job.
- **Data classification**: Labels data based on sensitivity, such as public, internal, confidential, or restricted.
- **Physical Security**: locked server rooms, badge access, surveillance

Real-Life Examples:
- Technical example: HTTPS protects confidentiality by encrypting data sent between a web browser and a website, so others on the network cannot easily read it.
- Everyday example: A locked filing cabinet protects paper documents by allowing only someone with the key to access them.

Common Threat:
A data breach violates confidentiality because an unauthorized person gains access to private or sensitive information.

Integrity

Definition:
Integrity means making sure information is accurate, complete, and not changed without permission. In cybersecurity, integrity protects data from being modified by attackers, software errors, insider misuse, or accidental mistakes.

A system with strong integrity allows users to trust that the information they see is correct and has not been secretly altered.
Tools such as digital signatures, checksums, hash functions, audit logs, input validation, version control, and permission controls help protect integrity.

Integrity matters because even a small unauthorized change can have serious consequences.
If a bank account balance, medical dosage, software update, or exam grade is changed without permission, the system may still be running, but it can no longer be trusted.

Integrity is not only about stopping changes; it is also about detecting changes when they happen and proving who made them. This is why many systems keep logs, timestamps, and records of user actions.

How it's protected?

- **Hashing**: Creates a fixed-length fingerprint of data so changes can be detected.
- **Digital signatures**: Prove that data came from a trusted sender and has not been modified.
- **Access permissions**: Limit who can edit, delete, or approve changes.
- **Audit logs**: Record important actions so suspicious or incorrect changes can be investigated.
- **Backups and version history**: Allow a system to restore correct data after corruption or unauthorized modification.
- **Checksums**: Verify file trasnfers haven't been corrupted or tampered with.

Real-Life Examples:

- Technical example: A digital signature verifies that a file or message has not been changed since it was signed by the original sender.
- Everyday example: A store receipt helps confirm that the items purchased and the amount paid match the actual transaction.

Common Threat:
A man-in-the-middle attack can violate integrity because an attacker may secretly intercept and alter messages while they travel between two parties

Availability

Definition:
Availability means ensuring that systems, services, and data are accessible when authorized users need them.
In cybersecurity, availability focuses on keeping websites, networks, applications, databases, and files running reliably.

A system with strong availability can continue operating during hardware failures, heavy traffic, cyberattacks, software bugs, power outages, or maintenance events.

Backups, redundancy, monitoring, disaster recovery plans, patch management, and

protection against service overloads all support availability.

Availability is important because a secure system is not useful if people cannot access it when they need it.
For example, an online banking system may protect customer data very well, but if customers cannot log in during an emergency, the system has failed in terms of availability.

Availability also includes performance and reliability, not just whether a system is completely online or offline.
A service that is technically running but extremely slow may still be considered unavailable for practical purposes.

How It's Protected?

- **Redundancy**: Uses extra servers, network devices, or storage so one failure does not stop the whole system.
- **Backups**: Keep copies of important data so it can be restored after deletion, corruption, or ransomware.
- **Load balancing**: Spreads user traffic across multiple servers to prevent overload.
- **Monitoring** and **alerts**: Detect problems early so administrators can respond quickly.
- **Disaster recovery planning**: Defines how an organization restores systems after major incidents.
- **Rate limiting**: Limits how many requests a user or system can send in a short time to reduce abuse.

Real-Life Examples:

- Technical example: A cloud service may use load balancing to spread traffic across multiple servers so the application remains available even when many users connect at once.
- Everyday example: Keeping a backup copy of an important document helps ensure the information is still available if the original is lost or damaged.

Common Threat:
A DDoS attack violates availability because it floods a service with too much traffic, making it slow or unreachable for legitimate users.

How They Work Together

Confidentiality, Integrity, and Availability must work together for a system to be considered secure.

A system that keeps data private but allows attackers to change it is not secure, because integrity is missing.
A system with accurate and private data is still not useful if users cannot access it when needed, because availability is missing.

Good cybersecurity balances all three principles so information stays private, trustworthy, and accessible to the right people at the right time.

In real systems, these principles often affect each other.
For example, strong encryption can improve confidentiality, but if the encryption keys are lost, availability may be harmed because users can no longer access the data.
Strict access controls can protect confidentiality and integrity, but if they are configured incorrectly, they may block legitimate users from doing their work.

Cybersecurity professionals must therefore design controls carefully so one principle is improved without unnecessarily weakening another.

A simple example is an online healthcare portal.
Confidentiality protects patient records from unauthorized viewing, **integrity** ensures that test results and prescriptions are accurate, and **availability** allows doctors and patients to access the portal when needed.
If any one of these fails, the whole system becomes less trustworthy and less secure.

Symmetric Encryption

Symmetric Encryption

What it is?

Symmetric encryption is a method of protecting data where the same secret key is used to encrypt and decrypt information.
The sender uses the key to turn readable data, called plaintext, into unreadable data, called ciphertext.
The receiver must have the same key to turn the ciphertext back into the original plaintext.

Symmetric encryption is called "symmetric" because both sides of the communication use the same shared secret. This makes it different from asymmetric encryption, where one key is public and another key is private.

Symmetric encryption is usually much faster than asymmetric encryption, so it is commonly used when a large amount of data needs to be protected, such as files, disks, database records, VPN traffic, or HTTPS session data.

The security of symmetric encryption depends heavily on protecting the secret key. If an attacker gets the key, they can usually decrypt the protected data.
Because of this, secure systems do not only focus on the encryption algorithm; they also focus on key generation, key storage, key rotation, and access control.

How it works?

1- The sender and receiver both have the same secret key.
2- The sender writes or prepares the original readable data, called plaintext.
3- The sender uses the secret key and an encryption algorithm to turn the plaintext into ciphertext.
4- The sender sends the ciphertext to the receiver.
5- The receiver uses the same secret key and the matching decryption process.
6- The ciphertext is converted back into the original plaintext.

In a real system, there are usually a few extra details.

The encryption process often uses a random value called an IV, nonce, or initialization value to help make repeated messages look different when encrypted.
This matters because encrypting the same message the same way every time could reveal

patterns to an attacker.

Modern encryption also often includes authentication, which means the receiver can detect if the ciphertext was modified before decryption.

Example flow:
Plaintext + Secret Key + Encryption Algorithm = Ciphertext
Ciphertext + Same Secret Key + Decryption Algorithm = Plaintext

For example, the message *__Transfer $100__* might become unreadable ciphertext like "__8fA92x...__" after encryption. Someone who intercepts the ciphertext should not be able to understand it without the secret key.

Common Algorithms:

Algorithm	Description	Key Size(s)	Current Usage Status
AES (Advanced Encryption Standard)	A modern, fast, and widely trusted encryption standard used to protect sensitive data.	128-bit, 192-bit, and 256-bit keys	Recommended and widely used today in secure systems, including HTTPS, VPNs, disk encryption, and file encryption.
DES (Data Encryption Standard)	An older encryption algorithm that was once a major standard but is now too weak.	56-bit key	Not considered secure today because its key size is too small and can be brute-forced with modern computing power.
3DES (Triple DES)	A stronger version of DES that applies DES encryption three times to increase security.	112-bit or 168-bit effective key strength, depending on configuration	Deprecated and being phased out because it is slower and less secure than modern algorithms like AES.
ChaCha20	A modern stream cipher designed to be fast, secure, and efficient, especially on devices without special AES hardware support.	256-bit key	Recommended and widely used today, often in secure network protocols such as TLS and VPN technologies.

More detail about these algorithms:

- **AES (Advanced Encryption Standard)**:
  AES is one of the most important symmetric encryption algorithms used today. It is a block cipher, meaning it encrypts data in fixed-size blocks rather than one character at a time.
  AES is trusted because it has been studied heavily by security experts and remains secure when implemented correctly. AES-128 is already considered strong for most uses, while AES-256 is often chosen for highly sensitive environments.

- **DES (Data Encryption Standard)**:
  DES was important historically, but its 56-bit key is now too short.
  A short key means an attacker can try every possible key using a brute-force attack. Because modern computers can test huge numbers of keys quickly, DES should not be used to protect sensitive data.

- **3DES (Triple DES)**:
  3DES was created to extend the life of DES by applying the DES process multiple times. It was stronger than original DES, but it is slow and has security limitations compared with modern options.
  Today, organizations are expected to migrate away from 3DES and use AES or another modern algorithm instead.

- **ChaCha20**:
  ChaCha20 is a stream cipher, meaning it encrypts data as a continuous stream rather than in fixed-size blocks. It is especially useful on mobile devices and systems where AES hardware acceleration is not available.
  ChaCha20 is commonly paired with Poly1305, a message authentication method, to provide both confidentiality and integrity.

Important Terms:

- Plaintext: The original readable data before encryption
- Ciphertext: The unreadable encrypted version of the data
- Key: A secret value used by the encryption algorithm to lock and unlock the data
- Encryption: The process of turning plaintext into ciphertext
- Decryption: The process of turning ciphertext back into plaintext
- Brute-force attack: An attack where someone tries many possible keys until the correct one is found
- Key rotation: The practice of replacing old keys with new keys on a regular schedule or after a security incident
- Nonce or IV: A value used during encryption to help make encrypted results unique, even when similar data is encrypted more than once

Block Ciphers and Stream Ciphers

Symmetric encryption algorithms are often grouped into block ciphers and stream ciphers. A block cipher encrypts data in fixed-size chunks, while a stream cipher encrypts data as a continuous flow. AES is a block cipher, while ChaCha20 is a stream cipher.

Block ciphers often need a mode of operation, which describes how each block of data is encrypted safely.
Examples include CBC, CTR, GCM, and XTS.
Modern systems often prefer authenticated encryption modes such as AES-GCM because they protect confidentiality and also help detect tampering.

Stream ciphers are useful for fast communication because they can encrypt data as it moves. However, they must be used carefully because reusing the same key and nonce combination can seriously weaken security.
This is why secure libraries are preferred over writing encryption code manually.

Real-World Use Cases:

- Disk encryption: Tools such as BitLocker, FileVault, and Linux LUKS use symmetric encryption to protect data stored on a device
- HTTPS session data: After a secure connection is established, HTTPS commonly uses symmetric encryption to protect the data exchanged between a browser and a website
- File encryption tools: Applications such as password managers, encrypted archives, and secure backup tools use symmetric encryption to protect files
- Database encryption: Organizations use symmetric encryption to protect sensitive database fields such as payment details, identification numbers, or health records
- VPN traffic: Virtual Private Networks use symmetric encryption to protect data traveling between a user's device and the VPN server
- Messaging applications: Secure messaging apps often use symmetric encryption for the actual message content after the users have established shared session keys

Symmetric encryption is popular in these cases because it is efficient.
Encrypting an entire hard drive, video stream, or large database with asymmetric encryption would usually be too slow.

In many real systems, asymmetric encryption is used first to securely agree on a shared secret key, and then symmetric encryption is used for the main data transfer.

Limitation:
The main limitation of symmetric encryption is the key distribution problem.
If two parties need to use the same secret key, they must find a secure way to share that key without an attacker intercepting it.

This problem becomes more difficult as the number of users grows.
For example, if two people need to communicate securely, they only need one shared key.
But if a company has thousands of users and systems, securely creating, sharing, storing, rotating, and revoking keys becomes much harder.

The key distribution problem is one reason modern security protocols often combine asymmetric and symmetric encryption.
For example, in HTTPS, asymmetric cryptography helps the browser and website securely agree on temporary session keys.
After that, symmetric encryption protects most of the actual data because it is faster and more efficient.

Best Practices

- Use modern, trusted algorithms such as AES-GCM or ChaCha20-Poly1305.
- Do not use outdated algorithms such as DES, and avoid 3DES in new systems.
- Generate keys using a secure random number generator.
- Store keys safely, such as in a key management system or hardware security module.
- Rotate keys when required by policy or after a suspected compromise.
- Do not hard-code secret keys directly inside source code.
- Use well-reviewed cryptographic libraries instead of creating custom encryption logic.

Summary

Symmetric encryption is one of the most widely used methods for protecting digital information. It is fast, efficient, and suitable for encrypting large amounts of data.
Its biggest challenge is safely sharing and managing the secret key, because anyone who has that key can decrypt the protected data.

The python code:

```python
# Import Fernet, a high-level symmetric encryption tool from the cryptography library.
from cryptography.fernet import Fernet

# Generate a random symmetric key; this same key is required for both encryption and decryption.
key = Fernet.generate_key()

# Create a Fernet object using the generated key so we can encrypt and decrypt data.
cipher = Fernet(key)

# Define the original plaintext message as a string so it is easy for humans to read.
plaintext_message = "This is a secret message for the cybersecurity internship project."

# Convert the plaintext string into bytes because Fernet encrypts byte data, not regular strings.
plaintext_bytes = plaintext_message.encode()

# Encrypt the plaintext bytes with the symmetric key, producing unreadable ciphertext bytes.
encrypted_message = cipher.encrypt(plaintext_bytes)

# Print the original readable message so we can compare it with the encrypted and decrypted versions.
print("Original message:", plaintext_message)

# Print the encrypted ciphertext; it appears unreadable because the data is protected.
print("Encrypted (ciphertext):", encrypted_message)

# Decrypt the ciphertext using the same symmetric key to recover the original plaintext bytes.
decrypted_bytes = cipher.decrypt(encrypted_message)

# Decode the decrypted bytes back into a normal string and print the recovered message.
print("Decrypted message:", decrypted_bytes.decode())
```

The cryptography library already installed

```
PS C:\Program Files\Microsoft VS Code> pip install cryptography
Requirement already satisfied: cryptography in c:\python314\lib\site-packages (48.0.0)
Requirement already satisfied: cffi>=2.0.0 in c:\python314\lib\site-packages (from cryptography) (2.0.0)
Requirement already satisfied: pycparser in c:\python314\lib\site-packages (from cffi>=2.0.0->cryptography) (3.0)

[notice] A new release of pip is available: 25.3 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Program Files\Microsoft VS Code>
```

The python code run:

```
PS C:\Program Files\Microsoft VS Code> & C:\Python314\python.exe c:/Users/Administrator/Desktop/Internship-2025/Internship-Projects/CyberSecurity/symmetric_demo.py

Original message: This is a secret message for the cybersecurity internship project.
Encrypted (ciphertext): b'gAAAAABqHEZgM1WdraBlJ9vSL4JhCDsvRlhVEUCXAUzxSQfcSpYPyaYdlDivvKsPpJ6-7CaspSHS8vO80y8zXNbmd7xI0WblN_B9mUW3zIB1gSiClPRbjGz6LoKjFK4DbRnAu6iD7
HTpmNwm-st_WW3d1lEjUET201HCDMvMYS8kyvv0_PRByZE='
Decrypted message: This is a secret message for the cybersecurity internship project.
PS C:\Program Files\Microsoft VS Code>
```

Asymmetric Encryption

Asymmetric Encryption

Definition:
Asymmetric encryption is a method of protecting data that uses a pair of related keys: a **public** key and a **private** key.
The public key can be shared with anyone and is used to encrypt messages, while the private key must be kept secret and is used to decrypt messages.
This design allows people to communicate securely even if they have never shared a secret key before.

Asymmetric encryption is also called public-key cryptography.
It is different from symmetric encryption because both sides do not need to already have the same secret key. Instead, one key is made public, and the other key stays private with its owner.

The public and private keys are mathematically connected, but it should be practically impossible to calculate the private key from the public key.
This means anyone can use Bob's public key to send Bob an encrypted message, but only Bob should be able to decrypt it with his private key.
This is the main idea that makes asymmetric encryption useful for secure communication over the internet.

How it works?
1- Bob generates a key pair made of a public key and a private key
2- Bob keeps his private key secret and protects it carefully
3- Bob shares his public key with Alice
4- Alice writes a plaintext message that she wants only Bob to read
5- Alice encrypts her message using Bob's public key
6- The encrypted message becomes ciphertext, which looks unreadable
7- Alice sends the ciphertext to Bob
8- Bob decrypts the ciphertext using his private key
9- The ciphertext turns back into the original plaintext message
10- Nobody intercepting the message can decrypt it without Bob's private key

Example flow:
Alice's plaintext + Bob's public key = Ciphertext
Ciphertext + Bob's private key = Alice's plaintext

The important point is that Bob's public key does not need to be hidden. Even if an attacker sees the public key, they should not be able to decrypt messages encrypted with it. The private key is the sensitive part and must never be shared.

Why It Solves the Key Distribution Problem?
Symmetric encryption has a key distribution problem because both parties need the same secret key before they can communicate securely.

If Alice wants to send Bob a symmetric key, she needs a secure channel to send it, but that creates a difficult question:
how can they create a secure channel before they already share a secret?
If an attacker intercepts the symmetric key while it is being shared, the attacker can decrypt future messages.

Asymmetric encryption helps solve this problem by removing the need to secretly share the decryption key. Bob can safely publish his public key, and Alice can use it to encrypt a message for him.
The only key that can decrypt the message is Bob's private key, which never has to leave Bob's device.

This does not mean public keys require no trust at all.
Alice must still be sure that the public key really belongs to Bob and not to an attacker pretending to be Bob.
Real systems solve this trust problem using certificates, certificate authorities, fingerprints, or key verification methods.

Common Algorithms

Algorithm	Brief Description	Typical Key Size	Primary Use Case
RSA	A widely used public-key algorithm based on the difficulty of factoring very large numbers.	2048-bit minimum is common today; 3072-bit or 4096-bit may be used for higher security.	Encrypting small data, exchanging keys, digital signatures, certificates, and older TLS/HTTPS systems.
ECC (Elliptic Curve Cryptography)	A modern family of public-key methods based on elliptic curve mathematics, offering strong security with smaller keys.	Common curves provide about 256-bit or 384-bit security parameters, such as P-256 or P-384.	TLS handshakes, mobile devices, digital signatures, cryptocurrency systems, and secure messaging.
ElGamal	A public-key encryption algorithm based on the discrete logarithm problem.	Often uses 2048-bit or larger parameters, depending on the group and security level.	Encryption systems, academic cryptography, and variants used in some secure communication designs.

More about these algorithms:

- **RSA**:
  RSA can be used for encryption and digital signatures. With RSA encryption, a public key encrypts data and the private key decrypts it. With RSA signatures, the private key signs data and the public key verifies the signature. RSA must be used with secure padding, such as OAEP for encryption or PSS for signatures, because plain RSA is not safe.

- **ECC (Elliptic Curve Cryptography)**:
  ECC provides strong security using much smaller keys than RSA. Smaller keys can mean faster operations, lower storage needs, and better performance on phones, embedded devices, and high-traffic servers. ECC is often used for key exchange and digital signatures, such as ECDH and ECDSA.

- **ElGamal**:
  ElGamal is important because it introduced ideas used in many later cryptographic systems. It can provide encryption based on mathematical problems that are hard to solve without the private key. However, ElGamal ciphertexts are larger than the original message, so it is less common in everyday applications than RSA or ECC.

Important Terms

- Public key: A key that can be shared openly and is used by others to encrypt messages or verify signatures.

- Private key: A secret key that must be protected and is used to decrypt messages or create signatures.
- Key pair: The public key and private key that are mathematically linked.
- Ciphertext: The unreadable encrypted version of a message.
- Plaintext: The original readable message before encryption.
- Padding: Extra structured data added during encryption to make the process secure against certain attacks.
- OAEP: Optimal Asymmetric Encryption Padding, a secure padding method commonly used with RSA encryption.
- Certificate: A digital document that helps prove a public key belongs to a specific website, person, or organization.

Real-World Use Cases

- HTTPS/TLS handshake: When a browser connects to a secure website, asymmetric cryptography helps verify the website's identity and securely establish shared session keys.

- SSH authentication: SSH can use public and private key pairs so a user can prove their identity to a server without typing a password every time.

- Email encryption with PGP: PGP allows someone to encrypt an email using the recipient's public key so only the recipient's private key can decrypt it.

Asymmetric encryption is usually not used to encrypt large files or long data streams directly.
Instead, it is often used at the beginning of a secure connection to exchange or agree on a symmetric session key.
After that, symmetric encryption protects the actual data because it is faster.

Limitation

Asymmetric encryption is slower than symmetric encryption because it uses more complex mathematical operations.
For example, RSA and ECC involve large-number mathematics that require more processing time than algorithms like AES or ChaCha20.
This makes asymmetric encryption inefficient for encrypting large files, full disks, database records, or high-volume network traffic directly.

Hybrid encryption solves this problem by combining both methods.

A system can use asymmetric encryption to securely exchange a temporary symmetric key, then use fast symmetric encryption to protect the actual data.
This is how many modern protocols work, including HTTPS/TLS: asymmetric cryptography helps set up trust and shared secrets, while symmetric encryption handles most of the communication.

Best Practices

- Protect private keys carefully and never share them.
- Use modern libraries and trusted protocols instead of writing custom cryptography.
- Use secure padding, such as OAEP for RSA encryption.
- Use recommended key sizes, such as at least 2048-bit RSA for basic modern security.
- Verify public keys using certificates, fingerprints, or trusted key directories.
- Rotate or replace keys if they are suspected to be compromised.
- Prefer modern algorithms and configurations recommended by current security standards.

Summary

Asymmetric encryption allows secure communication without requiring both parties to secretly share the same key in advance.
It uses a public key for encryption and a private key for decryption, which helps solve the key distribution problem found in symmetric encryption.
Because it is slower, real systems often use it together with symmetric encryption in a hybrid approach.

Python Code:

```python
# Import base64 so the encrypted bytes can be printed in a readable text format.
import base64

# Import RSA key generation support so we can create a public/private key pair.
from cryptography.hazmat.primitives.asymmetric import rsa

# Import OAEP and MGF1 padding tools, which make RSA encryption secure in practice.
from cryptography.hazmat.primitives.asymmetric import padding

# Import SHA-256 so OAEP can use a modern secure hash function.
from cryptography.hazmat.primitives import hashes

# Generate a 2048-bit RSA private key; this private key must be kept secret.
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048,
)

# Derive the matching public key from the private key; this public key can be shared.
public_key = private_key.public_key()

# Define the original plaintext message as a string so it is easy to read.
plaintext_message = "This is a secret message encrypted with Bob's public RSA key."

# Convert the plaintext string into bytes because cryptographic functions work with bytes.
plaintext_bytes = plaintext_message.encode()

# Encrypt the plaintext bytes with the public key so only the private key can decrypt them.
ciphertext = public_key.encrypt(
    plaintext_bytes,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None,
    ),
)

# Print the original readable message before encryption.
print("Original message:", plaintext_message)

# Encode the ciphertext with base64 so the encrypted bytes can be printed safely as text.
ciphertext_base64 = base64.b64encode(ciphertext).decode()

# Print the base64-encoded ciphertext; it should look unreadable compared with the plaintext.
print("Encrypted (base64):", ciphertext_base64)

# Decrypt the ciphertext with the private key using the same OAEP padding configuration.
decrypted_bytes = private_key.decrypt(
    ciphertext,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None,
    ),
)

# Decode the decrypted bytes back into a string and print the recovered plaintext message.
print("Decrypted message:", decrypted_bytes.decode())
```

Python run:

```
PS C:\Program Files\Microsoft VS Code> & C:\Python314\python.exe c:/Users/Administrator/Desktop/Internship-2025/Internship-Projects/CyberSecurity/asymmetric_demo.py
Original message: This is a secret message encrypted with Bob's public RSA key.
Encrypted (base64): bifrDT9QOrA6YiaWg5vsI0MsGxS6AxQRHtNf1rACcSTAfZTxlUldxyDQQlGBsAXSbaSi+0oRQPvZvssrrMnvqXmEA6s03PmV3oFWZ3ukTnEUNZHE8okDtKeqLLSKVTc69KBARFX2/E2snlKS6XRK3WcorkNaeLmzhIaUNUT27YMLh4ObMONh
yLTcox/N2Eo7Pdk8WYNFGaLHIjYt2WpjglCrSSYq/R1EdQEICh9GzHwVSORiEZKxns8s89aKtBVaRXCkJnTws4oMGO7ov2yH82BhNQtsfLq1Vk+WrIhNB7po+nBTG3qcugXbckg5ZIWk2+aMmu8lcJewAfw7NrjsQ==
Decrypted message: This is a secret message encrypted with Bob's public RSA key.
PS C:\Program Files\Microsoft VS Code>
```

Digital Signatures

Digital Signatures

What they are?
Digital signatures are a cryptographic method used to prove who sent or approved a message, file, or transaction.
They also prove that the signed data was not changed after it was signed.
Digital signatures do not encrypt the message content, so they do not hide the message from others; instead, they protect trust, identity, and tamper detection.

Digital signatures are based on asymmetric cryptography, which uses a private key and a public key.
The signer uses their private key to create the signature, and anyone with the signer's public key can verify that signature.
This is the opposite direction from public-key encryption, where a public key encrypts and a private key decrypts.

A simple way to understand this is: encryption protects secrecy, while digital signatures protect trust.
If Alice signs a document, Bob can check whether Alice's private key was used and whether the document changed after signing.
However, unless Alice also encrypts the document, Bob and others may still be able to read its contents.

What They Guarantee

- Integrity: The message was not modified after signing. If even one character changes, the signature verification should fail.
- Authenticity: The message was signed by the owner of the private key. If Bob verifies the signature with Alice's public key, he gains confidence that Alice's private key created it.
- Non-repudiation: The signer cannot reasonably deny having signed it, assuming their private key was secure and not stolen.

These guarantees are important because many security problems are not only about hiding information.

Sometimes the bigger question is whether information can be trusted.
For example, a software update may be public, but users still need to know it really came from the developer and was not modified by an attacker.

How They Work — Step-by-Step

1- Alice hashes her message
2- Alice encrypts the hash with her private key; this encrypted hash is the digital signature
3- Alice sends the original message and the signature to Bob
4- Bob decrypts the signature with Alice's public key and gets the original hash
5- Bob hashes the received message himself using the same hash algorithm
6- If the two hashes match, the signature is valid

Example flow:

Alice's message -> Hash function -> Message hash
Message hash + Alice's private key -> Digital signature
Message + Digital signature -> Sent to Bob

Bob hashes received message -> New hash
Bob verifies signature with Alice's public key -> Original signed hash
If both hashes match -> Valid signature

In real cryptographic libraries, the process is usually called "signing" and "verification" instead of manually saying "encrypting the hash" and "decrypting the signature."
This is because modern signature algorithms use secure padding and mathematical operations designed specifically for signatures.

For RSA signatures, PSS padding is commonly recommended because it is safer than older padding styles.

Why Hashing is Used

Digital signatures usually sign a hash of the message instead of signing the entire message directly.
A hash function takes data of any size and creates a fixed-size output called a digest.
For example, SHA-256 always produces a 256-bit digest, whether the input is a short sentence or a large file.

Hashing is useful for three main reasons:

- It is efficient because signing a short hash is faster than signing a large file

- It helps detect changes because a tiny change in the message creates a very different hash
- It gives the signature process a consistent input size

The messagesmay look similar to a human, but their hashes would be completely different, because the hash changes, the original signature would no longer verify.

Comparison: Encryption vs Digital Signatures

Topic	Encryption	Digital Signatures
Purpose	Hides message content from unauthorized readers.	Proves who signed the message and whether it changed.
Key used to "apply"	Usually the receiver's public key for asymmetric encryption, or a shared secret key for symmetric encryption.	The signer's private key.
Key used to "verify/decrypt"	The receiver's private key for asymmetric encryption, or the same shared secret key for symmetric encryption.	The signer's public key.
Guarantees confidentiality?	Yes, when encryption is used correctly.	No, signatures do not hide the message content.
Guarantees authenticity?	Not always by itself; extra authentication may be needed.	Yes, if the public key really belongs to the signer.

Real-World Use Cases

- Code signing: Software developers sign applications, drivers, and updates so users and operating systems can verify that the code came from a trusted publisher and was not modified
- SSL/TLS certificates: Websites use certificates signed by trusted certificate authorities to prove their identity during HTTPS connections
- PDF signing: Contracts, forms, and official documents can be digitally signed to show who approved them and whether the document changed afterward
- Cryptocurrency transactions: A user signs a transaction with their private key to prove they own the funds and approve the transfer

Common Threats and Mistakes

- Private key theft: If an attacker steals a private key, they can create signatures that appear to come from the real owner

- Unverified public keys: If Bob uses a fake public key that actually belongs to an attacker, Bob may trust forged signatures
- Weak hash algorithms: Old hash functions such as MD5 and SHA-1 are no longer recommended because attackers may be able to create collisions, where different data produces the same hash
- Signing the wrong data: A system must clearly define exactly what is being signed. If important fields are left out, attackers may modify the unsigned parts

Best Practices

- Keep private keys secret and store them securely
- Use modern algorithms and padding, such as RSA-PSS with SHA-256 or secure elliptic-curve signature algorithms
- Verify public keys through certificates, trusted directories, fingerprints, or another reliable method
- Avoid outdated hash functions such as MD5 and SHA-1
- Rotate or revoke keys if they are compromised
- Use trusted cryptographic libraries instead of creating signature logic manually

Summary

Digital signatures prove identity and protect data from unnoticed changes.
They provide integrity, authenticity, and non-repudiation, but they do not provide confidentiality because the message is not encrypted. In real systems, digital signatures are often combined with encryption so messages can be both private and trustworthy.

Python code:

```python
import base64

# Import InvalidSignature so we can catch the expected error for tampered data.
from cryptography.exceptions import InvalidSignature

# Import RSA key generation support so we can create a public/private key pair.
from cryptography.hazmat.primitives.asymmetric import rsa

# Import PSS and MGF1 padding, which are recommended for RSA signatures.
from cryptography.hazmat.primitives.asymmetric import padding

# Import SHA-256 so the signing and verification process uses a secure hash.
from cryptography.hazmat.primitives import hashes

# Generate a 2048-bit RSA private key; this private key is used to create signatures.
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048,
)

# Derive the public key from the private key; this public key is used to verify signatures.
public_key = private_key.public_key()

# Define the original plaintext message as a string so it is easy to read.
message = "Alice approves this cybersecurity internship document."

# Convert the message into bytes because cryptographic signing functions work with bytes.
message_bytes = message.encode()

# Sign the message bytes using the private key, RSA-PSS padding, and SHA-256 hashing.
signature = private_key.sign(
    message_bytes,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH,
    ),
    hashes.SHA256(),
# Print the original message so we can see exactly what was signed.
print("Message:", message)

# Encode the binary signature as base64 so it can be displayed safely as text.
signature_base64 = base64.b64encode(signature).decode()

# Print the base64 version of the signature.
print("Signature (base64):", signature_base64)

# Verify the signature with the public key to confirm the message is authentic and unchanged.
public_key.verify(
    signature,
    message_bytes,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH,
    ),
    hashes.SHA256(),
)

# If verification did not raise an exception, the signature is valid.
print("Signature valid: True")

# Modify the original message slightly to simulate tampering after the signature was created.
tampered_message = "Alice approves this cybersecurity internship documents."

# Convert the tampered message into bytes so it can be checked against the original signature.
tampered_message_bytes = tampered_message.encode()

# Try to verify the original signature against the tampered message.
try:
    # This should fail because the signature was created for the original message, not this modified one.
    public_key.verify(
        signature,
        tampered_message_bytes,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH,
        ),
        hashes.SHA256(),
    )

    # This line would only run if verification unexpectedly succeeded.
    print("Signature valid: True")
except InvalidSignature:
    # Catch the expected failure and explain that the message no longer matches the signature.
    print("Signature valid: False – message was tampered!")
```

Python code run:

```
PS C:\Program Files\Microsoft VS Code> & C:\Python314\python.exe c:/Users/Administrator/Desktop/Internship-2025/Internship-Projects/CyberSecurity/digital_signature_demo.py
Message: Alice approves this cybersecurity internship document.
Signature (base64): H1me2XGc/WMIhFGqzr3p+Y3ymLn3xL6/juYiVA4mLiyBMzHeWKF6YoYWvbwZHVWLP2E+tfPQYbuk/rhTzxqgvjyBBrkt0uODEzEo4csGBeBCdTdBOf506YDyUd2IL9Q8YPVneNZZwJY+eWKwP81dzOGO5AH+Mvw2WYOTyAXd//MsPEj6W8rL4
TQ98G7kPDJV61zCwUqCaALZZ/CXQfDhVV/jRtXjiTykkpR6YkYmCPQSv+PaBfamwZ8C1yBky12KIgv/U7a3wg+qO8FXd6fiZkKmIMZwhyM2mPaBar4VAPfB6ymXs85HyAVYAmeBdgiyPUsKmcLuZ23nanv4A7hjqQ==
Signature valid: True
Signature valid: False — message was tampered!
PS C:\Program Files\Microsoft VS Code>
```

Network Security Basics

Firewalls

What it is?
A firewall is a security control that monitors and filters network traffic based on rules. It acts like a checkpoint between networks, devices, or applications.
Firewalls can allow safe traffic, block suspicious traffic, and reduce exposure to attacks.

How it works:
A firewall checks network traffic as it enters or leaves a system or network.
It compares the traffic against rules, such as allowed IP addresses, blocked ports, or permitted applications.
If the traffic matches an allowed rule, it passes through; if it matches a blocked rule, it is denied.

Types or variants:

- **Packet filtering firewall**: Checks basic packet details such as source IP, destination IP, port number, and protocol
- **Stateful inspection firewall**: Tracks active connections and understands whether traffic belongs to a legitimate ongoing session
- **Application-layer firewall**: Looks deeper into application traffic, such as HTTP requests, instead of only checking IPs and ports
- **Web Application Firewall (WAF)**: A special application-layer firewall that protects web applications from attacks such as SQL injection and cross-site scripting

Why it matters?
Firewalls reduce the number of ways attackers can reach a system.
They help block unwanted traffic, protect internal networks, and enforce security policies.
For example, a company may allow public web traffic to a website but block direct database access from the internet.

Example or tool:
Windows Defender Firewall, Linux ufw, Linux iptables, Cloudflare WAF, AWS Security Groups.

VPNs

What it is?
A VPN, or Virtual Private Network, creates a protected connection between a user's device and another network
It is commonly used to securely access private company resources or to protect traffic on untrusted networks
A VPN can make a remote user appear as if they are connected from inside a private network

How it works?
1- The user connects to a VPN server using a VPN client
2- The VPN creates an encrypted tunnel between the user's device and the VPN server
3- Network traffic travels through this tunnel so outsiders cannot easily read it
4- The VPN server forwards the traffic to its destination, such as an internal company system or the public internet

The tunneling concept means the original traffic is wrapped inside another protected connection.
Encryption protects the contents of the traffic while it moves through the tunnel. This is especially useful on public Wi-Fi, where attackers may try to observe network traffic.

Types or variants:
- **Remote-access VPN**: Allows employees or users to connect securely to a private network from outside the office
- **Site-to-site VPN**: Connects two networks together, such as a company headquarters and a branch office
- **SSL/TLS VPN**: Uses TLS technology, similar to HTTPS, to protect VPN traffic
- **IPsec VPN**: A common VPN technology that protects IP traffic at the network layer
- **WireGuard**: A modern VPN protocol known for simplicity, speed, and strong cryptography
- **OpenVPN**: A widely used open-source VPN solution that supports secure remote access

Why it matters?
VPNs protect traffic from eavesdropping and help users securely access private systems from remote locations.
They are important for remote work because employees may need access to internal tools, file shares, or databases.
VPNs can also improve privacy on untrusted networks, although they do not make a user

completely anonymous.

Example or tool:
WireGuard, OpenVPN, Cisco AnyConnect, Fortinet FortiClient, Windows built-in VPN client

HTTPS

What it is?
HTTPS stands for Hypertext Transfer Protocol Secure. It is the secure version of HTTP, the protocol used by web browsers and websites to communicate.
HTTPS uses TLS, or Transport Layer Security, to encrypt traffic and help prove the identity of the website.

How it works?
1- A browser connects to a website using HTTPS
2- The website sends a digital certificate to prove its identity
3- The browser checks whether the certificate is trusted, valid, and issued for the correct domain name
4- The browser and server perform a TLS handshake to agree on secure session keys
5- After the handshake, most web traffic is protected with fast symmetric encryption

Plain HTTP is insecure because it sends data in readable form. Anyone who can observe the network may be able to see usernames, passwords, cookies, search queries, or private messages.
HTTP also does not strongly protect against tampering, so an attacker may be able to modify pages or inject malicious content

Type or variants:
- **HTTP**: Unencrypted web traffic; not safe for sensitive information
- **HTTPS with TLS**: Encrypted and authenticated web traffic; standard for modern websites
- **TLS certificates**: Digital certificates that help prove a website's identity
- **Certificate authorities**: Trusted organizations that issue and sign certificates for websites.

Why it matters?
HTTPS protects confidentiality by encrypting web traffic, integrity by detecting tampering, and authenticity by helping users connect to the real website.
It is essential for login pages, banking, online shopping, email, healthcare portals, and any website that handles user data.

Example or tool:
Browser padlock icon, Let's Encrypt certificates, OpenSSL, curl https://example.com

Port Scanning

What it is?
Port scanning is the process of checking a system to see which network ports are open, closed, or filtered.
A port is a numbered communication endpoint used by services on a computer.
For example, web servers often use port 80 for HTTP and port 443 for HTTPS.

How it works?
A scanner sends network connection attempts or special packets to a target host and observes the response.

- If a service responds, the port may be open
- If the connection is refused, the port may be closed
- If there is no response, a firewall may be filtering the traffic

Open ports matter because they show which services are reachable.
Attackers scan ports to find exposed services they may be able to exploit.
Defenders scan their own systems to discover unexpected services, verify firewall rules, and reduce the attack surface.

Types or variants:

- **TCP connect scan**: Attempts a full TCP connection to check whether a port is open
- **SYN scan**: Sends a TCP SYN packet and checks the response without completing the full connection
- **UDP scan**: Checks UDP services, which can be slower and harder to interpret
- **Version detection**: Attempts to identify the software and version running on an open port

Why it matters?
Port scanning helps security teams understand what is exposed on a system or network.
It can reveal unnecessary services, outdated software, or accidental exposure of internal tools.
Knowing open ports is important for both attackers and defenders, but defenders use this information to fix weaknesses before attackers exploit them.

Example or tool:
Nmap, Masscan, Netcat, Python socket scripts for simple local checks.

Nmap

What it is?
Nmap, short for Network Mapper, is a popular open-source tool used for network discovery and security auditing.
It can identify live hosts, open ports, running services, software versions, and sometimes operating system details.
Nmap is widely used by system administrators, cybersecurity professionals, and penetration testers.

How it works?
Nmap sends carefully crafted network probes to a target and analyzes the responses.
Based on those responses, it determines whether ports are open, closed, or filtered.
With extra options, it can also try to identify service versions, run scripts, or detect operating system information.

Types or variants:
- **Basic scan**: Checks common ports on a target
- **Version detection**: Attempts to identify service names and versions
- **Specific port scan**: Checks only selected ports instead of scanning many ports
- **Script scan**: Uses the Nmap Scripting Engine to perform deeper checks

Why it matters?
Nmap helps defenders understand their network from an attacker's point of view. It can reveal exposed services, misconfigured systems, or software that needs patching.
It is powerful, so it should only be used on systems you own or have permission to test.

Example or tool:
in bash: nmap 127.0.0.1
This performs a basic scan of localhost, which means the computer running the command.

nmap -sV 127.0.0.1
This performs version detection and tries to identify what serives and versions are running on open ports.

nmap -p 80,443 127.0.0.1
This scans only ports 80 and 443 on localhost, which are commonly used for HTTP and HTTPS.

Summary Table

Topic	Purpose	Common Tool/Protocol
Firewalls	Filter traffic and block unwanted network access.	Windows Defender Firewall, `ufw`, WAF
VPNs	Create encrypted tunnels for secure remote or private network access.	WireGuard, OpenVPN, IPsec
HTTPS	Protect web traffic with encryption, integrity, and website identity verification.	TLS, certificates, Let's Encrypt
Port Scanning	Identify open ports and exposed services.	Nmap, Netcat, Python `socket`
Nmap	Discover hosts, ports, services, and versions for security auditing.	`nmap`

Python Code:

```python
# Import the built-in socket library so we can attempt TCP connections without external dependencies.
import socket

# Define the scan target as localhost, meaning this script only checks the current machine.
target_host = "127.0.0.1"

# Define the specific ports we want to check for common local services.
ports_to_scan = [22, 80, 443, 3306, 8080, 8443]

# Map each port number to a friendly service name so the output is easier to understand.
service_names = {
    22: "SSH",
    80: "HTTP",
    443: "HTTPS",
    3306: "MySQL",
    8080: "HTTP-Alt",
    8443: "HTTPS-Alt",
}

# Keep a counter of open ports so we can print a useful summary at the end.
open_count = 0

# Loop through each port in the list and test whether a TCP connection succeeds.
for port in ports_to_scan:
    # Look up the friendly service name for this port, or use "Unknown" if it is not listed.
    service_name = service_names.get(port, "Unknown")

    # Create a TCP socket using a context manager so it closes automatically after each check.
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
        # Set a short timeout so closed or filtered ports do not make the script wait too long.
        client_socket.settimeout(0.5)

        try:
            # Attempt to connect to the target host and port; success usually means the port is open.
            client_socket.connect((target_host, port))

            # Count the open port so the final summary is accurate.
            open_count += 1

            # Print the open result using the required output format.
            print(f"[OPEN]   Port {port} — {service_name}")
        except (ConnectionRefusedError, socket.timeout, TimeoutError, OSError):
            # Treat refused connections, timeouts, and socket errors as closed for this beginner demo.
            print(f"[CLOSED] Port {port} — {service_name}")

# Count how many ports were checked so the summary can show the total scan size.
total_checked = len(ports_to_scan)

# Print the final scan summary with the number of open ports found.
print(f"Scan complete. {open_count} port(s) open out of {total_checked} checked.")
```

Python code run:

```
PS C:\Program Files\Microsoft VS Code> & C:\Python314\python.exe c:/Users/Administrator/Desktop/Internship-2025/Internship-Projects/CyberSecurity/port_ping_demo.py
[CLOSED] Port 22 — SSH
[CLOSED] Port 80 — HTTP
[CLOSED] Port 443 — HTTPS
[CLOSED] Port 3306 — MySQL
[CLOSED] Port 8080 — HTTP-Alt
[CLOSED] Port 8443 — HTTPS-Alt
Scan complete. 0 port(s) open out of 6 checked.
PS C:\Program Files\Microsoft VS Code>
```

Final Report

Cybersecurity Internship — Final Report

Intern: Mediane Ozeir
Organization: XpertNurse
Duration: 3 months
Focus Area: Cybersecurity Fundamentals

Introduction

This internship project focused on building a beginner-friendly but practical foundation in core cybersecurity concepts.
The purpose was to understand how security principles are applied to protect data, systems, and users in real-world environments.
Instead of only reading definitions, the project connected theory with small Python demonstrations that showed how encryption, decryption, signing, verification, and basic port checking work.

The main topics explored were the CIA Triad, symmetric encryption, asymmetric encryption, digital signatures, and network security basics.
These topics are important because they appear in almost every area of cybersecurity, from secure websites and VPNs to software updates and healthcare data protection.

Each task helped explain a different part of the security picture: how to keep information private, how to prove it was not changed, how to verify identity, and how to reduce network exposure.

The overall learning goal was to understand the basic building blocks that more advanced cybersecurity topics depend on.
By the end of the project, the focus was not only on memorizing terms, but also on understanding why these concepts matter and how they work together in real systems.
This foundation is especially relevant to healthcare technology, where protecting sensitive patient and organizational data is a major responsibility.

Topic Summaries

1. **CIA Triad**

What I learned: The CIA Triad is a core cybersecurity model made of Confidentiality, Integrity, and Availability.
Confidentiality means keeping information private and limiting access to authorized users only.
Integrity means making sure information stays accurate, complete, and protected from unauthorized changes.
Availability means ensuring systems and data are accessible when authorized users need them.

Key takeaway: A system is not truly secure unless confidentiality, integrity, and availability are all protected together.

2. **Symmetric Encryption**

What I learned: Symmetric encryption uses the same secret key for both encryption and decryption.
It is fast and efficient, which makes it useful for protecting large amounts of data such as files, disk storage, VPN traffic, and HTTPS session data.

I learned about algorithms such as AES, DES, 3DES, and ChaCha20, including why modern systems prefer AES and ChaCha20 while older options like DES are no longer secure.
I also learned that the main weakness of symmetric encryption is the key distribution problem, because both sides must safely share the same secret key.

Key takeaway: Symmetric encryption is powerful and fast, but its security depends on protecting and sharing the secret key safely.

Demo built: I created **symmetric_demo.py**, which uses Fernet from the cryptography library to generate a symmetric key, encrypt a plaintext message, and decrypt it back to the original text.

3. **Asymmetric Encryption**

What I learned: Asymmetric encryption uses a public key and a private key instead of one shared secret key.
A public key can be shared openly and used to encrypt data, while the private key must be kept secret and used to decrypt it.

This helps solve the key distribution problem because two people do not need to secretly share the same key before communicating.
I also learned about algorithms such as RSA, ECC, and ElGamal, and why asymmetric encryption is usually slower than symmetric encryption.

Key takeaway: Asymmetric encryption makes secure communication possible between parties that have not already shared a secret key.

Demo built: I created **asymmetric_demo.py**, which generates a 2048-bit RSA key pair, encrypts a message with the public key using OAEP padding and SHA-256, then decrypts it with the private key.

4. **Digital Signatures**

What I learned: Digital signatures prove who signed a message and whether the message was changed after signing.
They provide integrity, authenticity, and non-repudiation, but they do not encrypt or hide the message content.

I learned that signatures use asymmetric key pairs in a different way from encryption: the private key signs, and the public key verifies.
I also learned why hashing is part of the process, because a signature is usually created over a hash of the message rather than the full message directly.

Key takeaway: Digital signatures do not protect secrecy; they protect trust.

Demo built: I created **digital_signature_demo.py**, which signs a message with an RSA private key using PSS padding and SHA-256, verifies it with the public key, then shows that verification fails if the message is tampered with.

5. **Network Security Basics**

What I learned: Network security focuses on protecting communication between systems and reducing exposure to attacks.

I learned how firewalls filter traffic, how VPNs create encrypted tunnels, how HTTPS protects web traffic with TLS, and why plain HTTP is insecure.
I also learned what ports are and why open ports matter to both attackers and defenders.

Finally, I studied Nmap as a common tool for discovering open ports, services, and versions during authorized security testing.

Key takeaway: Network security tools help control what can communicate, what is exposed, and how safely data travels across networks.

Demo built: I created **port_ping_demo.py**, a localhost-only Python script using the built-in socket library to check selected TCP ports and report whether they are open or closed.

Connections Between Concepts

All five topics are connected because they support the same overall goal: protecting systems and data.

The CIA Triad is the foundation because confidentiality, integrity, and availability explain what security controls are trying to achieve.

Symmetric encryption supports confidentiality by protecting data quickly and efficiently, while asymmetric encryption helps solve the problem of securely exchanging keys between parties that do not already share a secret.

In real systems such as HTTPS, both methods are often combined through hybrid encryption: asymmetric cryptography helps establish trust and session keys, then symmetric encryption protects the actual data in transit.

Digital signatures also use asymmetric key pairs, but their purpose is different because they prove authenticity and integrity rather than hiding content.

Network security tools connect these ideas to real communication systems, where firewalls reduce unwanted access, VPNs encrypt traffic, and HTTPS protects web sessions.
Port scanning and Nmap help defenders understand which services are exposed so they can reduce risk before attackers take advantage of weaknesses.

Together, these concepts show that cybersecurity is not one single tool, but a layered approach where each control supports a different part of secure system design.

Personal Reflection

This project was not extremely difficult, but the most challenging part was understanding how similar concepts differ from each other, especially encryption versus digital signatures.

At first, it was easy to think that anything involving keys was doing the same job, but the demos made the difference much clearer.

The most interesting part was learning how HTTPS combines multiple ideas, including certificates, asymmetric encryption, symmetric encryption, and integrity protection.

In a healthcare context like XpertNurse, these concepts matter because patient information, staff accounts, medical records, and internal systems all need to stay private, accurate, and available.
Even basic mistakes, such as weak key handling or unnecessary open ports, could create real risks for healthcare data.

This foundation prepares me to study more advanced topics such as secure system design, vulnerability assessment, incident response, cloud security, and healthcare compliance.

Project File Structure

File	Type	Description
`cia_triad.md`	Markdown report	Explains Confidentiality, Integrity, and Availability with examples, threats, and how the principles work together.
`symmetric_encryption.md`	Markdown report	Explains symmetric encryption, common algorithms, use cases, limitations, and best practices.
`symmetric_demo.py`	Python demo	Demonstrates symmetric encryption and decryption using Fernet from the `cryptography` library.
`asymmetric_encryption.md`	Markdown report	Explains public/private key encryption, key distribution, common algorithms, use cases, and hybrid encryption.
`asymmetric_demo.py`	Python demo	Demonstrates RSA encryption with a public key and decryption with a private key using OAEP and SHA-256.

digital_signatures.md	Markdown report	Explains digital signatures, integrity, authenticity, non-repudiation, hashing, and real-world uses.
digital_signature_demo.py	Python demo	Demonstrates RSA signing and verification, including a failed verification after message tampering.
network_security.md	Markdown report	Covers firewalls, VPNs, HTTPS, port scanning, Nmap, and their security benefits.
port_ping_demo.py	Python demo	Scans selected localhost ports using Python's built-in socket library and prints open or closed results.
final_report.md	Final report	Summarizes the internship project, connects the concepts, and reflects on the learning outcomes.